```
        {
          "match_phrase_prefix": {
            "overview": "4K Ultra HD"
          }
        }],
      "minimum_should_match": 1
    }
  }
}
```

Matches at least one leaf query in the should clause

In the listing, we set `minimum_should_match` to 1. This means the query tries to match the criteria defined in the leaf queries of the `should` clause but with the condition that at least one of the queries must be a positive match: if a product is silver *or* the product is 4K Ultra HD *or* the price is between $500 and $2,000. However, if none of the criteria in the `should` clause match, the query fails, as we are asking the query to satisfy the `minumum_should_match` parameter.

As you may have guessed, a `bool` query can be declared with a `should` clause on its own—that is, without a `must` clause. Based on whether a `bool` query consists of a `must` clause along with the `should` clause, the default value of `minimum_should_match` varies. The default value of `minimum_should_match` is set to 0 if the `bool` query consists of a `should` with a `must` clause; it is set to 1 with only a `should` clause (see table 11.3).

Table 11.3 Default values of the `minimum_should_match` attribute

Clauses	Default value of `minimum_should_match`
`should` clause only (no `must` clause)	1
`should` with a `must` clause	0

So far, we've looked at `must`, `must_not`, and `should` clauses. Although `must_not` runs in a filter context, `must` and `should` run in a query context. We discussed the query context versus filter context in chapter 8, but for completeness, let's touch base on these concepts here. Queries run in a query context execute the appropriate relevancy algorithm, so we can expect relevancy scores associated with the resultant documents. Queries in a filter context do not output scores and are performant because execution of the scoring algorithm isn't needed. That leads to a clause that doesn't use relevance scoring but works in a filter context: the `filter` clause.

11.3.7 The filter clause

The `filter` clause fetches all documents that match the criteria, similar to the `must` clause. The only difference is that the `filter` clause runs in a filter context, so the results are not scored. Remember, running a query in a filter context speeds up query performance because it caches the query results returned by Elasticsearch. The following listing shows a `filter` query in action.